MEHRAN UNIVERSITY OF ENGINEERING & TECHNOLOGY JAMSHORO
DEPARTMENT OF ELECTRONIC ENGINEERING
COURSE: FPGA BASED DIGITAL DESIGN (PR)
BATCH =20ES, 6TH SEMESTER

COURSE TEACHER: ENGR. SHOAIB HASSAN

PBL ACTIVITY

Instructions

- *The solution must be submitted by each student individually.*
- *The answers should be submitted as printed hardcopy.*
- *The softcopy of the report should also be submitted on MS Team.*
- *All references to books, online resources, lectures, and other material must be properly included in the report.*

PBL activity Description: Design a system using NEXYS4 DDR FPGA board which performs all the mentioned Arithmetic, Logic, Shift, rotate and relational functions given in table 1 between two signals A(3:0) and B(3:0). The block diagram of the system is shown in the following figure 1.
Use all eight 7-segment displays of the FPGA board and display the outputs as given table 2.

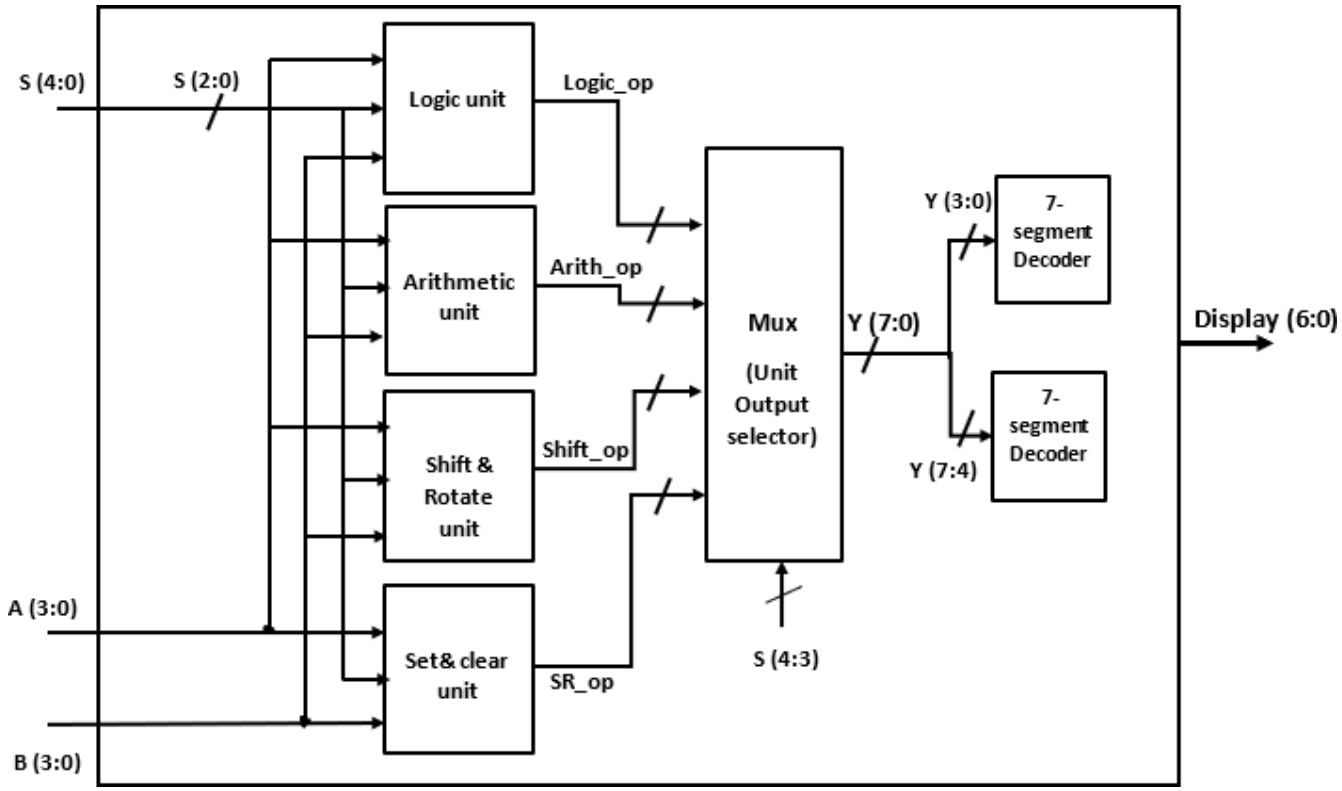


Figure 1.

PBL Tasks

1. Attach the VHDL code in structural Style for designing this system with description of the code.
2. Simulate the system by writing Test Bench.
3. Attach the simulation result and hardware results snapshots.

Table 1.

Select Input (S)	Operation	Function	Unit
00000	Y<= B	Transfer B	**Arithmetic**
00001	Y<= A	Transfer A	
00010	Y<= B+1	Increment B	
00011	Y<= B-1	Decrement B	
00100	Y<= A+1	Increment A	
00101	Y<= A-1	Decrement A	
00110	Y<= A+B	Add A and B	
00111	Y<= B-A	Subtract A from B	
01000	Y<=A and X"00"	Clear A	**Set and Clear**
01001	Y<= B xor X"FF"	Toggle B	
01010	Y<=A OR X"FF"	Set A	
01011	Y<=B and X"00"	Clear B	
01100	Y<= A xor X"FF"	Toggle A	
01101	Y<=A OR X "FF"	Set B	
01110	Y<= inp2 & inp1	Concatenate inp2 and inp1	
01111	Y<= Y	No change	
10000	Y<= NOT B	Complement B	**Logic**
10001	Y<= NOT A	Complement A	
10010	Y<= A AND B	AND	
10011	Y<= A OR B	OR	
10100	Y<= A NAND B	NAND	
10101	Y<= A NOR B	NOR	
10110	Y<= A XOR B	XOR	
10111	Y<= A XNOR B	XNOR	
11000	Y<= A SLL 2	A logical left shift by 2	**Shift & rotate**
11001	Y<= B SRL 2	B logical right shift by 2	
11010	Y<= B SLA 3	B arithmetic left shift by 2	
11011	Y<= A SRA 1	B arithmetic right shift by 2	
11100	Y<= B ROR 2	B rotate right by 2	
11101	Y<= A ROL 4	A Rotate Left by 2	
11101	Y<= A SLL -2	A logical left shift by -2	
11111	Y<= B SRL -2	B logical right shift by -2	

Table 2.

OUTPUTS								
S.No.	**Display8**	**Display7**	**Display6**	**Display5**	**Display4**	**Display3**	**Display2**	**Display1**
1.	d	E	P	⊦	E	L	E	C
2.	b	A	⊦	C	h	-	2	0
3.	r	O	L	L	n	O	R1	R2
4.	r	E	S	⊦	-	Y-digit3	Y-digit2	Y-digit1

Note: For Shift and rotate operations, use ***Concatenation Operator (&).***

Where R1 and R2 represent LSD and MSD of your Roll Number, respectively. In the last row of the table above, Y-digit1 to Y-digit3 must be in decimal format. The results in the table will change every 10 seconds.

Assessment Rubrics

Performance Indicators	Marks	PLO Mapping	Good	Average	Poor	Secured Marks
Problem Recognition	0.5	PLO-5	Demonstrates the ability to identify problems	Demonstrates the ability to identify problems with some assistance	Not able to identify any problems.	
Innovation	0.8	PLO-5	Demonstrates an Innovative idea and successfully implements it	Demonstrates an Innovative idea but doesn't know how to implement it	Fails to give any innovative idea	
Use of Modern Engineering Tools	0.7	PLO-5	Hardware/software are extensively used in the course	Hardware/software are somewhat utilized, effort was put into learning new software	Hardware/softw are are not utilized, no attempt was made at learning new software	
Calculation and Coding	01	PLO-5	Calculation / Coding is complete and details are provided.	Calculation / Coding is complete but details are missing.	Calculation / Coding is incomplete	
Observation/ Program results	01	PLO-5	Observations / program results are complete and details are provided.	Observations / program results are complete but details are missing.	Observations / Program results are incomplete.	
Teamwork	01	PLO-10	Actively engages and cooperates with other group members in an effective manner.	Cooperates with other group members in a reasonable manner.	Distracts or discourages other group members from conducting the experiment.	

Code :-

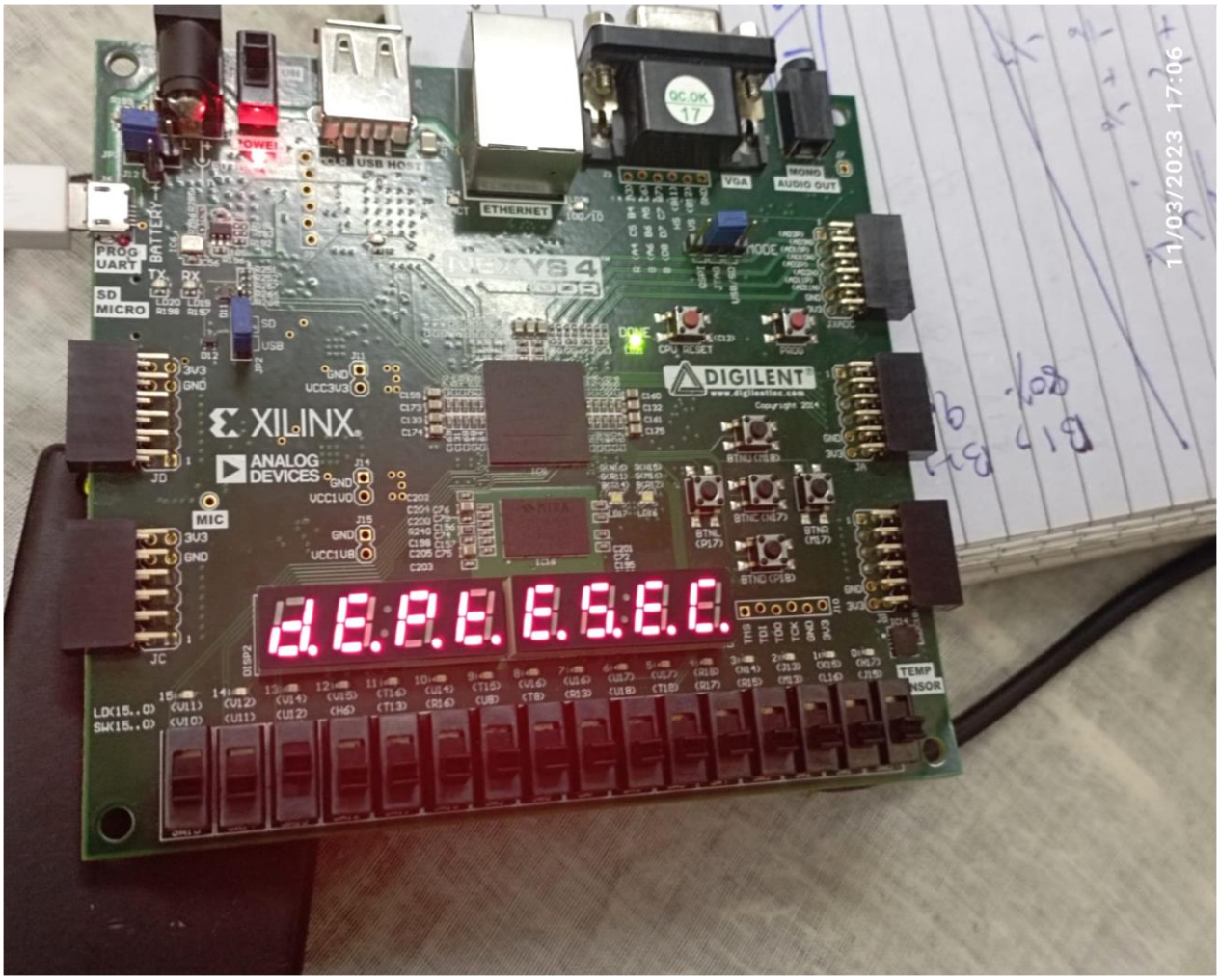
```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
use IEEE.NUMERIC_STD.ALL;
entity pbl62 is
    Port ( Clk : in  STD_LOGIC;
in1 : in  STD_LOGIC_VECTOR (3 downto 0);
           in2 : in  STD_LOGIC_VECTOR (3 downto 0);
           Sel : in  STD_LOGIC_VECTOR (4 downto 0);
 Anode : out  STD_LOGIC_VECTOR (7 downto 0);
           Display : out  STD_LOGIC_VECTOR (7 downto 0));
end pbl62;


architecture pbl62 of pbl62 is


CONSTANT R1: std_logic_vector(3 DOWNTO 0):= "1001"; ---9
CONSTANT R2: std_logic_vector(3 DOWNTO 0):= "0001"; ---1
Signal A, B : std_logic_vector(7 downto 0);
Signal logic,Shift_rotate,Set_clear : std_logic_vector(7 downto 0);
Signal Arith : Signed(7 downto 0);
SIGNAL A_sig, B_sig: SIGNED (7 DOWNTO 0 );
Signal Slow : std_logic;
SIGNAL fast : std_logic;
Signal Temp ,sevensegcounter : std_logic_vector(3 downto 0);
SIGNAL y_logicdecimal: integer range 0 to 255;
SIGNAL y_arithdecimal: integer;
SIGNAL y_SRdecimal: integer range 0 to 255;
SIGNAL y_SCdecimal: integer range 0 to 255;
SIGNAL ydigit1: integer;
SIGNAL ydigit2: integer;
SIGNAL ydigit3: integer;
SIGNAL ydigit1conv,ydigit2conv,ydigit3conv: std_logic_vector(3 downto 0);
SIGNAL y1,y2,y3: STD_LOGIC_VECTOR(7 DOWNTO 0);
signal roll, batch, dept : std_logic;
begin
process(in1,in2)
begin
A<= in1 & "0110";
B<=in2 & "0011";
end process;
----LOGIC UNIT---
With Sel(2 downto 0) Select
Logic <=  (not B) when "000",
       (not A) when "001",
 (A and B) when "010",
 (A or B) when "011",
 (A nand B) when "100",

 (A nor B) when "101",
 (A xor B) when "110",
  (A xnor B) when others;

A_sig <= SIGNED(A);
B_sig <= SIGNED(B);
```

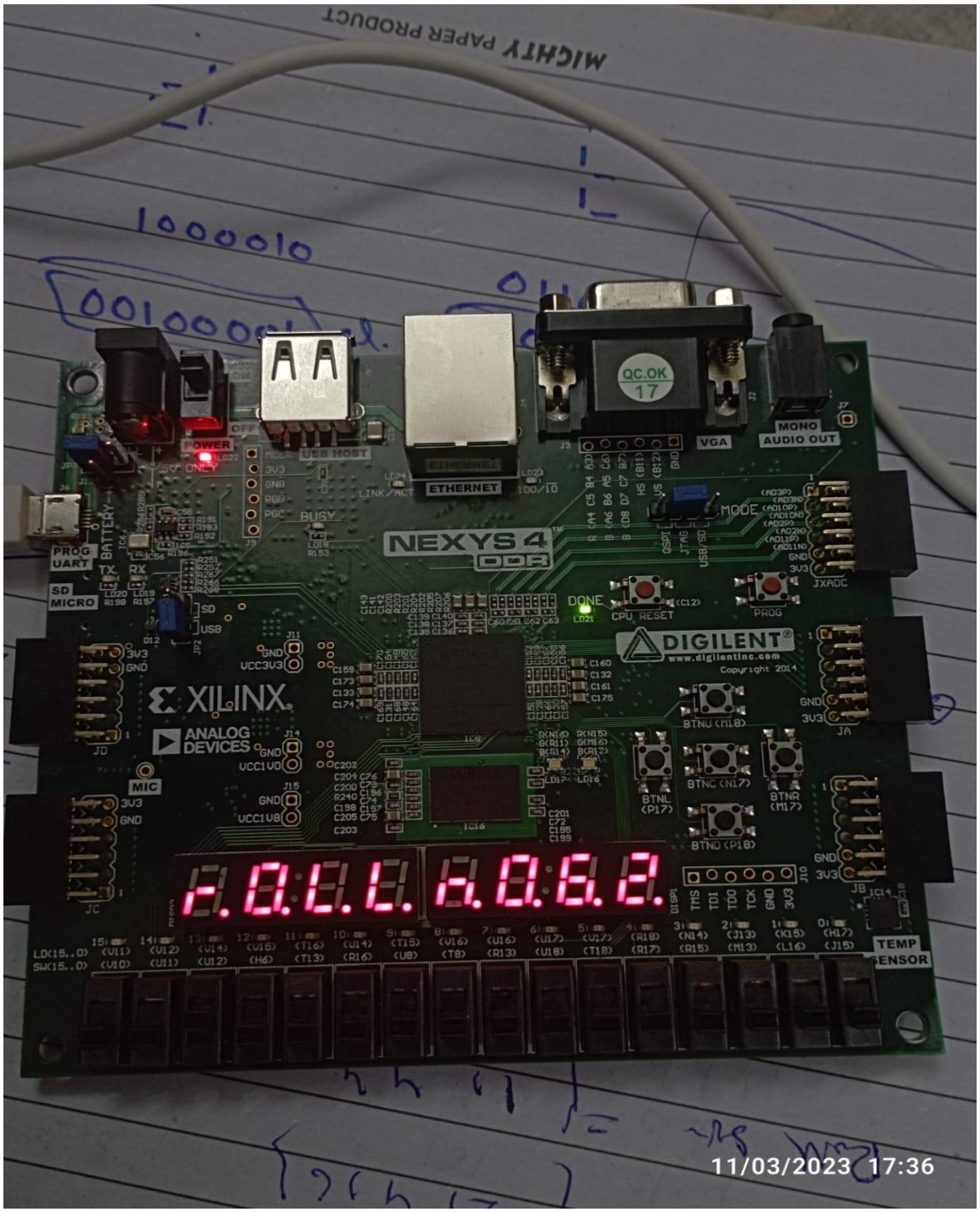
```
With Sel(2 downto 0) Select
Arith<= B_sig when "000",
   A_sig when "001",
 (B_sig + 1) when "010",
 (B_sig - 1) when "011",
 (A_sig + 1) when "100",
 (A_sig - 1) when "101",
 (A_sig + B_sig) when "110",
 (B_sig - A_sig) when others;


With Sel(2 downto 0) Select
Shift_rotate <= (A(5 downto 0) & "00") when "000",
          ("00" & B(7 downto 2)) when "001",
 (B(4 downto 0) & B(0) & B(0) & B(0)) when "010",
 (A(7) & A(7 downto 1)) when "011",
 (B(1 downto 0) & B(7 downto 2)) when "100",
 (A(3 downto 0) & A(7 downto 4)) when "101",
 ("00" & A(7 downto 2)) when "110",
 (B(5 downto 0)& "00") when others;



With Sel(2 downto 0) Select
Set_clear <= (A and X"00") when "000",
          (B xor X"FF") when "001",
     (A or X"FF") when "010",
     (B and X"00") when "011",
     (A xor X"FF") when "100",
     (A or X"FF") when "101",
     (in2 & in1) when "110",
     set_clear when others;

 process(clk,slow)
 variable divider: integer range 0 to 500;
 begin
 if(clk'event and clk='1') then
 divider:=divider+1;
 if(divider=500) then
 slow<= not slow;
 divider:=0;
 end if;
 end if;
 end process;


 process(clk, fast)
    variable divider : integer range 0 to 100000000;
begin
    if (rising_edge(clk)) then
        divider := divider + 1;
        if (divider = 100000000) then
            fast <= not fast;
            divider := 0;
        end if;
    end if;
end process;
```

```
 Process(slow)
 begin
 if(slow'event and slow='1') then
 Temp <= Temp+1;
 end if;
 end process;

 Process(slow)
 begin
 if(slow'event and slow='1') then
 sevensegcounter <= sevensegcounter+1;
 end if;
 end process;

with Temp Select
Anode <="11111110" WHEN "0000",
 "11111101" WHEN "0001",
 "11111011" WHEN "0010",
 "11110111" WHEN "0011",
 "11101111" WHEN "0100",
 "11011111" WHEN "0101",
 "10111111" WHEN "0110",
 "01111111" WHEN "0111",
 "11111111" WHEN OTHERS;
 y_SCdecimal <= to_integer(unsigned(Set_clear));
 y_SRdecimal <= to_integer(unsigned(Shift_rotate));
 y_logicdecimal <= to_integer(unsigned(logic));
 y_arithdecimal <= to_integer(signed(Arith));

 WITH Sel(4 DOWNTO 3) SELECT
 ydigit1 <= y_logicdecimal mod 10 WHEN "00",
y_arithdecimal mod 10 WHEN "01",
y_SRdecimal mod 10 WHEN "10",

y_SCdecimal mod 10 WHEN OTHERS;
 WITH Sel(4 DOWNTO 3) SELECT
 ydigit2 <= y_logicdecimal/10 mod 10 WHEN "00",
y_arithdecimal/10 mod 10 WHEN "01",
y_SRdecimal/10 mod 10 WHEN "10",
y_SCdecimal/10 mod 10 WHEN OTHERS;
 WITH Sel(4 DOWNTO 3) SELECT
 ydigit3 <= y_logicdecimal/100 mod 10 WHEN "00",
y_arithdecimal/100 mod 10 WHEN "01",
y_SRdecimal/100 mod 10 WHEN "10",
y_SCdecimal/100 mod 10 WHEN OTHERS;

 ydigit1conv <= std_logic_vector(to_unsigned(ydigit1,ydigit1conv'length));
 ydigit2conv <= std_logic_vector(to_unsigned(ydigit2,ydigit2conv'length));
 ydigit3conv <= std_logic_vector(to_unsigned(ydigit3,ydigit3conv'length));

 WITH ydigit1conv SELECT
y1<= "11111001" when "0001",
        "10100100" when "0010",
        "10110000" when "0011",
        "10011001" when "0100",
        "10010010" when "0101",
        "10000010" when "0110",
        "11111000" when "0111",
        "10000000" when "1000",
```

```
        "10010000" when "1001",
        "11000000" when "0000",
        "11111111" when others;
 WITH ydigit2conv SELECT
y2<= "11111001" when "0001",
        "10100100" when "0010",

        "10110000" when "0011",
        "10011001" when "0100",
        "10010010" when "0101",
        "10000010" when "0110",
        "11111000" when "0111",
        "10000000" when "1000",
        "10010000" when "1001",
        "11000000" when "0000",
        "11111111" when others;
 WITH ydigit3conv SELECT
y3<= "11111001" when "0001",
        "10100100" when "0010",
        "10110000" when "0011",
        "10011001" when "0100",
        "10010010" when "0101",
       "10000010" when "0110",
        "11111000" when "0111",
        "10000000" when "1000",
        "10010000" when "1001",
        "11000000" when "0000",
        "11111111" when others;
WITH sevensegcounter SELECT
 Display<="00100100" WHEN "0000",
"00000010" WHEN "0001",
"01000000" WHEN "0010",
"00101011" WHEN "0011",
"01000111" WHEN "0100",
"01000111" WHEN "0101",
"01000000" WHEN "0110",
"00101111" WHEN "0111",
"11111111" WHEN OTHERS;

end Architecture pbl62;
```

Hardware :-


11/03/2023 17:06
QC OK
17
ETHERNET
VGA
AUDIO OUT
USB HOST
PROG UART
SD MICRO
BATTERY
DIGILENT
XILINX
ANALOG DEVICES
MIC
JD
JC
LD(15..0)
SW(15..0)
TEMP SENSOR


NEXYS 4
DDR
DIGILENT
www.digilentinc.com
Copyright 2014
XILINX
ANALOG DEVICES
ETHERNET
VGA
MONO AUDIO OUT
USB HOST
POWER
PROG UART
SD MICRO
MIC
TEMP SENSOR
CPU_RESET
PROG
DONE
BTNU (N18)
BTNC (N17)
BTNL (P17)
BTNR (M17)
BTND (P18)
A.0.E.E.A.0.6.2.
11/03/2023 17:36